

Giới thiệu về Java

① Java là gì?

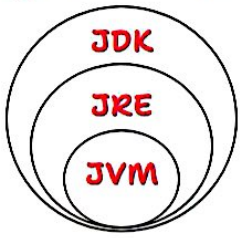
Java là ngôn ngữ lập trình hướng đối tượng, bậc cao, dựa trên lớp (class), được Sun Microsystems phát triển (nay thuộc Oracle). Java độc lập nền tảng, bảo mật, mạnh mẽ và được dùng rộng rãi để xây dựng ứng dụng doanh nghiệp, ứng dụng Android, ứng dụng web và nhiều hơn nữa.



② Đặc điểm của Java

- ✓ Đơn giản
- ✓ Hướng đối tượng (OOP)
- ✓ Độc lập nền tảng
- ✓ Bảo mật
- ✓ Mạnh mẽ (Robust)
- ✓ Đa luồng
- ✓ Trung lập kiến trúc
- ✓ Khả chuyển
- ✓ Hiệu năng **cao**
- ✓ Phân tán
- ✓ Động (Dynamic)
- ✓ Tự động quản lý bộ nhớ (GC)

③ JDK vs JRE vs JVM



JDK (Java Development Kit)

Cung cấp công cụ để phát triển như javac, java, debugger, v.v.

JRE (Java Runtime Environment)

Cung cấp thư viện, JVM và các thành phần khác để chạy chương trình Java.

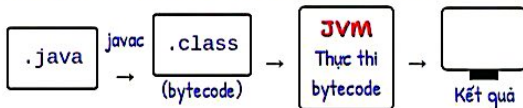
JVM (Java Virtual Machine)

Thực thi bytecode và cung cấp môi trường chạy.

Thành phần	Bao gồm
JDK	JRE + JVM + Công cụ phát triển (javac, jar, javadoc, v.v.)
JRE	JVM + Thư viện lõi + Các thành phần runtime khác
JVM	Class Loader + Bytecode Verifier + Execution Engine + Runtime Data Areas

④ Java hoạt động như thế nào?

Mã Java (.java) được biên dịch thành bytecode (.class) bằng trình biên dịch Java (javac). Bytecode này được JVM thực thi trên mọi nền tảng.



⑤ Luồng biên dịch & thực thi

1. **Viết** mã Java (.java)
- ↓
2. Biên dịch bằng **javac** (Compiler)
- ↓
3. Sinh ra **bytecode** (file .class)
- ↓
4. **JVM** nạp và kiểm tra bytecode
- ↓
5. **JVM** thực thi bytecode
- ↓
6. Kết quả được hiển thị

⑥ Chương trình Java đầu tiên ★

```
// Chương trình Hello World
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, Java!");
    }
}
```

- từ khóa **class** → định nghĩa một lớp
- **main()** → điểm bắt đầu của chương trình
- **System.out.println()** → in ra kết quả
- Mọi câu lệnh đều kết thúc bằng ;
- Tên file phải trùng tên class (HelloWorld.java)

✓ Ghi nhớ

- Java là "Write Once, Run Anywhere" (WORA).
- JVM giúp Java độc lập nền tảng.
- Bytecode là trái tim của Java.

★ Góc phỏng vấn

Java do James Gosling thiết kế năm 1995 và phát hành lần đầu năm 1996. Phần mở rộng của mã nguồn là .java, còn bytecode là .class

⚠ Lỗi thường gặp

- Tên file và tên class không khớp.
- Thiếu phương thức **main()**.
- Nhầm lẫn giữa biên dịch và thực thi.

1 JDK (Java Development Kit)

- Bộ công cụ đầy đủ để phát triển Java.
- Bao gồm JRE + các công cụ phát triển.
- Dùng để viết, biên dịch và chạy chương trình Java.

Bao gồm:

- JRE
- Trình biên dịch (javac)
- Công cụ (javadoc, jar, jdb, v.v.)

2 JRE (Java Runtime Environment)

- Cung cấp thư viện, JVM và các thành phần khác để chạy ứng dụng Java.
- Không bao gồm công cụ phát triển.

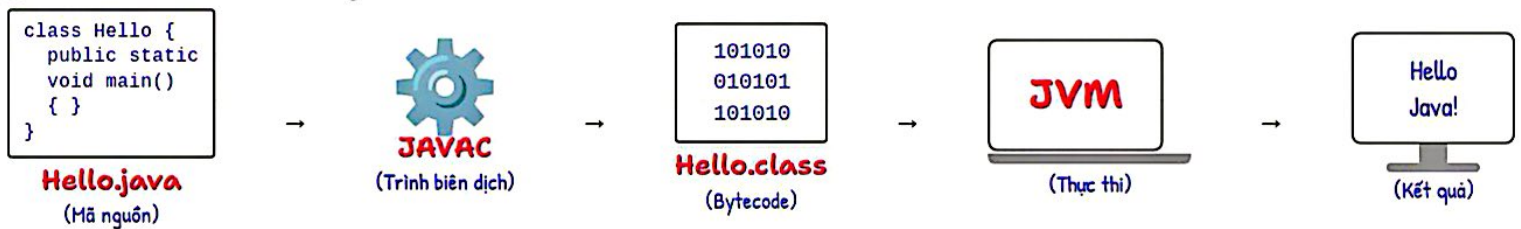
Bao gồm:

- JVM
- Thư viện lõi (rt.jar, v.v.)

3 JVM (Java Virtual Machine)

- Trái tim của Java.
- Thực thi Java bytecode.
- Độc lập nền tảng - Write Once, Run Anywhere.
- Quản lý bộ nhớ, luồng, bảo mật và nhiều hơn nữa.

4 Quá trình biên dịch



5 Bytecode

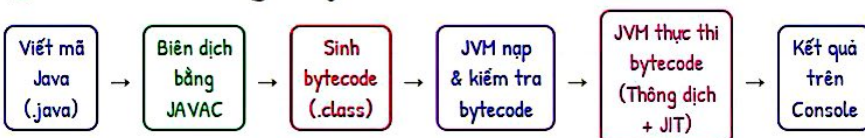
- Kết quả đầu ra của trình biên dịch Java (javac).
- Độc lập nền tảng.
- Được JVM thực thi, không chạy trực tiếp trên hệ điều hành.
- Phần mở rộng file: **.class**



Lưu ý quan trọng

JVM chuyển bytecode thành mã máy ngay tại thời điểm chạy nhờ trình biên dịch JIT (Just-In-Time) để có hiệu năng tốt hơn.

6 Sơ đồ luồng thực thi



🎯 Điểm cốt lõi

- ✓ Phát triển trong **JDK**
- ✓ Chạy trong **JRE**
- ✓ Thực thi diễn ra bên trong **JVM**
- ✓ Write Once, Run Anywhere

★ Java độc lập nền tảng vì JVM có sẵn trên mọi hệ điều hành phổ biến.

① Biến là gì?

Biến là một vùng nhớ có tên dùng để lưu trữ dữ liệu, và giá trị của nó có thể thay đổi trong quá trình chương trình chạy.

```
int age = 25;
```

↑ ↑ ↑
Kiểu dữ liệu Tên biến Giá trị

② Kiểu dữ liệu

a) Kiểu nguyên thủy (Primitive)

Lưu một giá trị đơn lẻ.

Kiểu	Kích thước	Giá trị mặc định
byte	1 byte	0
short	2 bytes	0
int	4 bytes	0
long	8 bytes	0L
float	4 bytes	0.0f
double	8 bytes	0.0d
char	2 bytes	'\u0000'
boolean	1 bit	false

b) Kiểu tham chiếu

Lưu tham chiếu tới đối tượng.

- String
- Array (Mảng)
- Class
- Interface
- Object
- Collection (List, Set, Map)
- v.v.

③ Literal (hằng giá trị)

- Số nguyên : 25, -10, 0
- Số thực : 3.14, -0.5f
- Ký tự : 'A', 'z', '\n'
- Chuỗi : "Hello Java"
- Boolean : true, false

④ Quy tắc đặt tên biến

- ✓ Phải bắt đầu bằng chữ cái, dấu '\$' hoặc '_'
- ✓ Có thể chứa chữ cái, chữ số, '_' và '\$'
- ✓ Phân biệt hoa thường: age và Age là hai biến khác nhau
- ✓ Không được trùng từ khóa Java (int, class, v.v.)
- ✗ Ví dụ tên không hợp lệ: 1age, int, @age

⑤ Bộ nhớ hoạt động ra sao

	Bộ nhớ	
1001 →	25	→ <code>int age = 25;</code>
1002 →	3.14	→ <code>double pi = 3.14;</code>
1003 →	'A'	→ <code>char grade = 'A';</code>
1004 →	true	→ <code>boolean isActive = true;</code>
	...	



Lưu ý:

Mỗi biến chiếm bộ nhớ tùy theo kiểu dữ liệu. Kiểu nguyên thủy lưu giá trị thật, còn kiểu tham chiếu lưu địa chỉ trỏ tới đối tượng trong bộ nhớ.

★ Góc phòng vấn

```
int a = 10;  
int b = a;     // b nhận một bản sao giá trị  
a = 20;       // b không thay đổi  
System.out.println(b);   // Kết quả: 10
```

★ Thực hành tốt

- ✓ Đặt tên biến có ý nghĩa.
- ✓ Khai báo biến gần nơi sử dụng.
- ✓ Dùng final cho hằng số.
- ✓ Tuân theo quy ước đặt tên camelCase.

① Toán tử số học

Dùng để thực hiện các phép toán cơ bản.

Toán tử	Mô tả	Ví dụ
+	Cộng	$a + b$
-	Trừ	$a - b$
*	Nhân	$a * b$
/	Chia	a / b
%	Chia lấy dư	$a \% b$
++	Tăng 1 đơn vị	$a++$
--	Giảm 1 đơn vị	$a--$

③ Toán tử logic

Dùng để kết hợp nhiều điều kiện.

Toán tử	Mô tả	Ví dụ
&&	VÀ (AND)	$(a > 0) \&\& (b > 0)$
	HOẶC (OR)	$(a > 0) \ \ (b > 0)$
!	PHỦ ĐỊNH (NOT)	$!(a > 0)$

⑤ Toán tử một ngôi

Thao tác trên một toán hạng duy nhất.

Toán tử	Mô tả	Ví dụ
+	Dấu dương	$+a$
-	Dấu âm	$-a$
++	Tăng trước	$++a$
--	Giảm trước	$--a$
!	Phủ định Logic	$!true \rightarrow false$
~	Đảo bit	$\sim a$

② Toán tử quan hệ

So sánh hai giá trị, kết quả là boolean (true/false).

Toán tử	Mô tả	Ví dụ
==	Bằng	$a == b$
!=	Khác	$a != b$
>	Lớn hơn	$a > b$
<	Nhỏ hơn	$a < b$
>=	Lớn hơn hoặc bằng	$a >= b$
<=	Nhỏ hơn hoặc bằng	$a <= b$

④ Toán tử gán

Dùng để gán giá trị cho biến.

Toán tử	Ví dụ	Tương đương
=	$a = b$	$a = b$
+=	$a += b$	$a = a + b$
-=	$a -= b$	$a = a - b$
*=	$a *= b$	$a = a * b$
/=	$a /= b$	$a = a / b$
%=	$a \% = b$	$a = a \% b$

⑥ Ép kiểu (Type Casting)

Chuyển đổi từ kiểu dữ liệu này sang kiểu dữ liệu khác.

- Ngầm định (Nới rộng) - diễn ra tự động.
- Tường minh (Thu hẹp) - phải làm thủ công.

1) Ngầm định (Widening)

Kiểu nhỏ $\rightarrow$ Kiểu lớn

Ví dụ:

```
int a = 10;
```

```
double b = a; // int  $\rightarrow$  double
```

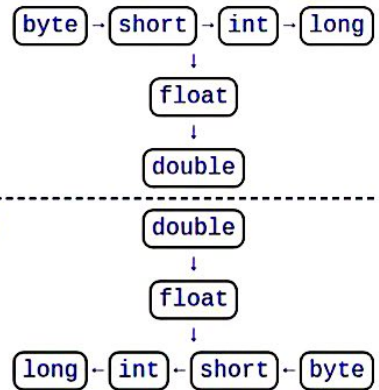
2) Tường minh (Narrowing)

Kiểu lớn $\rightarrow$ Kiểu nhỏ (thủ công)

Ví dụ:

```
double x = 25.75;
```

```
int y = (int) x; // double  $\rightarrow$  int
```



💡 Ví dụ minh họa

```
int a = 10, b = 3;
```

```
System.out.println("a + b = " + (a + b)); // 13
```

```
System.out.println("a - b = " + (a - b)); // 7
```

```
System.out.println("a * b = " + (a * b)); // 30
```

```
System.out.println("a / b = " + (a / b)); // 3
```

```
System.out.println("a % b = " + (a % b)); // 1
```

```
System.out.println("a > b = " + (a > b)); // true
```

★ Góc phỏng vấn

Toán tử là viên gạch tạo nên biểu thức và điều kiện trong Java. Hãy nắm rõ độ ưu tiên của toán tử và ép kiểu để tránh lỗi.

⚠️ Lỗi thường gặp

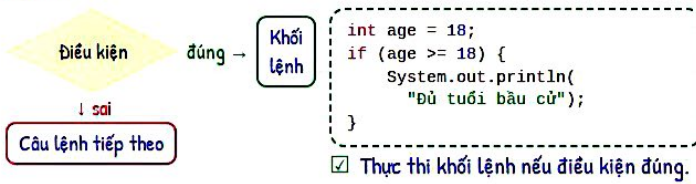
Luôn cẩn thận khi ép kiểu. Việc thu hẹp kiểu (narrowing) có thể làm mất mát dữ liệu.

Câu lệnh điều khiển

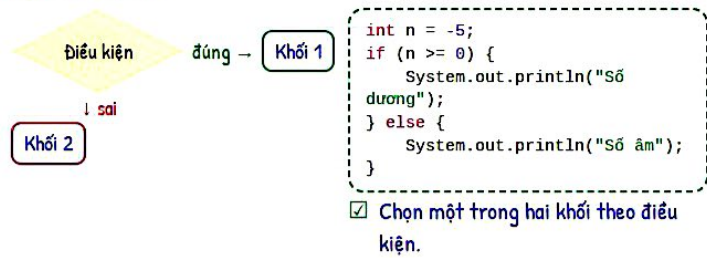
5

Câu lệnh điều khiển làm thay đổi luồng thực thi dựa trên điều kiện, hoặc lặp lại một khối lệnh nhiều lần.

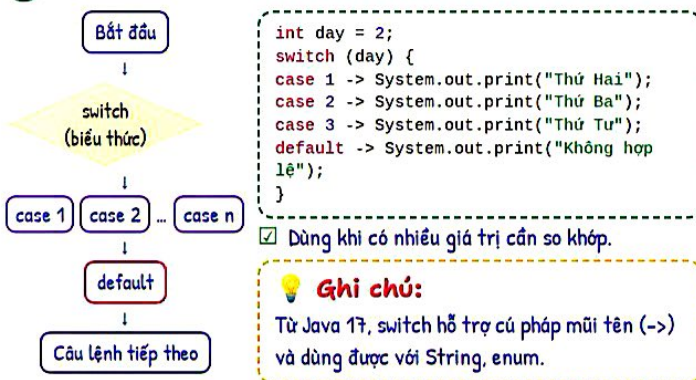
1 if



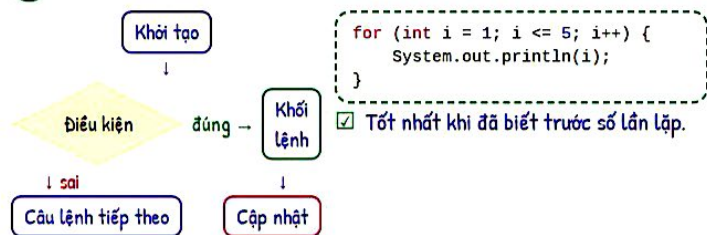
2 if-else



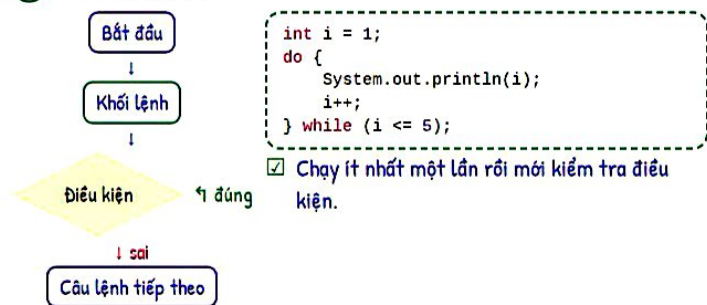
3 switch



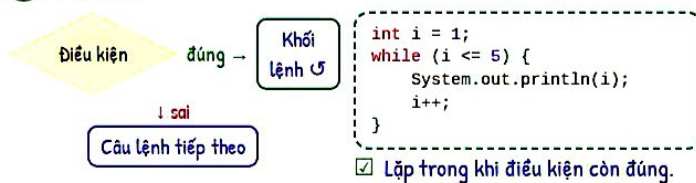
4 for



6 do-while



5 while



7 break & continue

• break

Thoát khỏi vòng lặp hoặc switch ngay lập tức.

```
for (int i = 1; i <= 5; i++) {
    if (i == 3) break;
    System.out.print(i + " ");
}
```

★ Kết quả: 1 2

• continue

Bỏ qua lần lặp hiện tại và chuyển sang lần kế tiếp.

```
for (int i = 1; i <= 5; i++) {
    if (i == 3) continue;
    System.out.print(i + " ");
}
```

★ Kết quả: 1 2 4 5

🔗 **Góc phỏng vấn:** Chọn đúng câu lệnh điều khiển giúp code dễ đọc và chạy hiệu quả hơn.

☑ **Thực hành tốt:** Tránh lồng quá sâu, dùng return sớm, break hoặc continue để đơn giản hóa logic.

Phương thức trong Java

6

Phương thức (method) là một khối lệnh thực hiện một nhiệm vụ cụ thể.

① Cú pháp phương thức

```
accessModifier returnType methodName(parameterList) {  
    // thân phương thức  
    return value;  
}
```

↑ ↑ ↑ ↑
public, private int, void, Tên của Giá trị
protected, v.v. String, v.v. phương thức đầu vào

② Tham số (Parameters)

- Tham số dùng để truyền dữ liệu vào phương thức.
- Chúng đóng vai trò là chỗ giữ chỗ cho giá trị thật.

```
public int add(int a, int b) {  
    return a + b;  
}
```

↳ a và b là các tham số

③ Kiểu trả về (Return Type)

- Phương thức có thể trả giá trị về cho nơi gọi bằng câu lệnh return.

Kiểu trả về	Ý nghĩa
void	Không trả về giá trị
int, double...	Trả về giá trị số
String	Trả về chuỗi văn bản
boolean	Trả về true hoặc false

Ví dụ

```
public int  
square(int n){  
    return n * n;  
}
```

④ Nạp chồng phương thức (Overloading)

- Cùng tên phương thức nhưng khác danh sách tham số.
- Giúp code dễ đọc hơn.

```
int add(int a, int b) {  
    return a + b;  
}  
  
int add(int a, int b, int c) {  
    return a + b + c;  
}
```

⑤ Đệ quy (Recursion)

- Là phương thức tự gọi lại chính nó.
- Dùng để giải bài toán phức tạp theo cách đơn giản.

```
int factorial(int n) {  
    if (n == 0) return 1;  
    return n * factorial(n - 1); // gọi  
    đệ quy  
}
```

factorial(3)

↓

3 - factorial(2)

↓

2 - factorial(1)

↓

1 - factorial(0)

↓

1

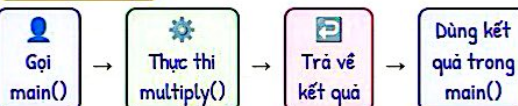
⑥ Phạm vi của biến (Scope)

Loại biến	Phạm vi	Thời gian tồn tại	Ví dụ
Biến cục bộ	Trong phương thức hoặc khối lệnh	Tạo ra khi phương thức / khối được gọi	int x = 10;
Biến thể hiện (non-static)	Trong class, ngoài các phương thức	Tồn tại khi đối tượng còn tồn tại	int age;
Biến static (biến của lớp)	Toàn bộ class	Tồn tại đến khi chương trình kết thúc	static int count;

⑦ Ví dụ hoàn chỉnh

```
public class Demo {  
    public static int multiply(int a, int b) {  
        return a * b;  
    }  
    public static void main(String[] args) {  
        int result = multiply(5, 4); // gọi phương thức  
        System.out.println("Kết quả: " + result);  
    }  
}
```

Cách hoạt động?



Góc phòng vấn

Phương thức giúp tái sử dụng code, dễ bảo trì và lập trình theo hướng module hóa.

★ Thực hành tốt

- Đặt tên phương thức có ý nghĩa.
- Giữ phương thức ngắn gọn, tập trung một việc.

- Tránh lặp code.
- Tuân theo nguyên tắc "một trách nhiệm".

📌 Ghi nhớ:

Một phương thức tốt giúp code của bạn sạch sẽ và dễ hiểu.

Mảng trong Java

7

Mảng là tập hợp các phần tử cùng kiểu dữ liệu được lưu trên các ô nhớ liên tiếp nhau.

① Mảng 1 chiều

- Lưu các phần tử cùng kiểu trên một hàng.
- Chỉ số (index) bắt đầu từ 0.

Ví dụ:

```
int[] numbers = new int[5];  
numbers[0] = 10;  
numbers[1] = 20;  
numbers[2] = 30;  
numbers[3] = 40;  
numbers[4] = 50;
```

Biểu diễn trong bộ nhớ

Index 0 1 2 3 4

10	20	30	40	50
----	----	----	----	----

↑
numbers
int[]

② Mảng 2 chiều

- Là mảng của các mảng (dòng và cột).
- Hữu ích cho bảng, ma trận, lưới.

Ví dụ:

```
int[][] matrix = new int[2][3];  
matrix[0][0] = 1; matrix[0][1] = 2; matrix[0][2] = 3;  
matrix[1][0] = 4; matrix[1][1] = 5; matrix[1][2] = 6;
```

Biểu diễn trong bộ nhớ

Cột →	0	1	2	
Dòng ↓				
0	1	2	3	matrix int[][]
1	4	5	6	

③ Các thao tác với mảng

Thao tác	Mô tả	Ví dụ
Khai báo	Tạo một mảng	<code>int[] arr = new int[5];</code>
Truy cập	Lấy phần tử theo chỉ số	<code>int val = arr[2];</code>
Cập nhật	Sửa phần tử tại một chỉ số	<code>arr[2] = 100;</code>
Độ dài	Lấy tổng số phần tử	<code>int len = arr.length;</code>
Duyệt mảng	Lặp qua tất cả phần tử	<code>for (int i = 0; i < arr.length; i++) { ... }</code>

④ Vòng lặp for-each

- Cách duyệt mảng đơn giản hơn.
- Chỉ đọc (không thay đổi được chỉ số).

Ví dụ:

```
int[] arr = {10, 20, 30, 40, 50};  
for (int num : arr) {  
    System.out.println(num);  
}
```

★ Kết quả

10
20
30
40
50

⑤ Lớp tiện ích Arrays

- `java.util.Arrays` cung cấp nhiều phương thức hữu ích để làm việc với mảng.

Phương thức	Mô tả
<code>Arrays.toString(arr)</code>	Trà về chuỗi biểu diễn mảng
<code>Arrays.sort(arr)</code>	Sắp xếp mảng tăng dần
<code>Arrays.fill(arr, val)</code>	Điền toàn bộ mảng bằng một giá trị
<code>Arrays.copyOf(arr, len)</code>	Sao chép mảng sang độ dài mới
<code>Arrays.equals(a1, a2)</code>	So sánh hai mảng

💡 Góc phỏng vấn

- Mảng có kích thước cố định.
- `IndexOutOfBoundsException` xảy ra khi chỉ số không hợp lệ.
- Dùng `for-each` để duyệt mảng cho gọn.

✅ Thực hành tốt

- Đặt tên mảng có ý nghĩa.
- Luôn kiểm tra chỉ số trước khi truy cập.
- Ưu tiên dùng các phương thức của lớp `Arrays`.
- Không lưu dữ liệu lớn trên stack, hãy dùng mảng trên heap (qua `new`).

Chuỗi (String) trong Java

8

① String

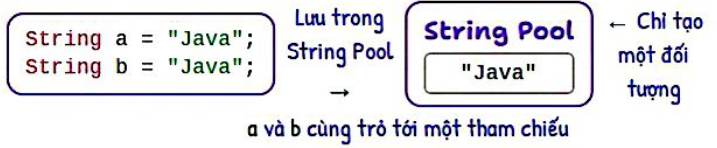
- String là một dãy các ký tự.
- Trong Java, String là đối tượng của lớp `java.lang.String`.
- Được dùng rộng rãi để lưu và xử lý văn bản.

Ví dụ:

```
String name = "Java";
String msg = new String("Hello");
System.out.println(name); // Java
System.out.println(msg);  // Hello
```

② String Pool

- Các chuỗi literal được lưu trong **String Pool**.
- Nếu literal đã tồn tại, Java trả về cùng một tham chiếu.
- Giúp tối ưu bộ nhớ.



③ String là bất biến (Immutable)

- Đối tượng String **không thể thay đổi** sau khi tạo.
- Mọi thao tác "sửa" chuỗi thực chất đều tạo ra một đối tượng mới.

```
String s1 = "Hello";
String s2 = s1.concat(" World");
System.out.println(s1); // Hello
System.out.println(s2); // Hello World
```

s1 không đổi. Một đối tượng mới được tạo cho s2.

④ StringBuilder

- Lớp **có thể thay đổi**, dùng để chỉnh sửa chuỗi.
- Không đồng bộ → không an toàn với đa luồng.
- Hiệu quả hơn String khi chạy đơn luồng.

```
StringBuilder sb = new StringBuilder("Java");
sb.append(" Programming");
sb.insert(5, " is");
sb.delete(0, 1); // xóa 'J'
System.out.println(sb); // ava is Programming
```

⑤ StringBuffer

- Lớp **có thể thay đổi**, tương tự StringBuilder.
- Được đồng bộ → an toàn với đa luồng.
- Chậm hơn StringBuilder nhưng an toàn khi đa luồng.

```
StringBuffer sb = new StringBuffer("Java");
sb.append(" Programming");
sb.reverse();
System.out.println(sb); // gnimmargorP avaJ
```

⑥ String vs StringBuilder vs StringBuffer

Tiêu chí	String	StringBuilder	StringBuffer
Thay đổi được?	Không	Có	Có
An toàn đa luồng?	Có	Không	Có
Hiệu năng	Chậm	Nhanh	Chậm
Trường hợp dùng	Ít chỉnh sửa	Đơn luồng, cần hiệu năng	Môi trường đa luồng

⑦ Các phương thức String thường dùng

Phương thức	Mô tả	Ví dụ	Kết quả
<code>length()</code>	Trả về độ dài chuỗi	<code>"Java".length()</code>	4
<code>charAt(index)</code>	Trả về ký tự tại vị trí index	<code>"Java".charAt(1)</code>	a
<code>substring(begin, end)</code>	Cắt một phần chuỗi [begin, end)	<code>"Java".substring(1, 3)</code>	av
<code>equals(str)</code>	So sánh nội dung	<code>"Java".equals("java")</code>	false
<code>equalsIgnoreCase(str)</code>	So sánh, bỏ qua hoa thường	<code>"Java".equalsIgnoreCase("java")</code>	true
<code>contains(str)</code>	Kiểm tra có chứa chuỗi con	<code>"Java Programming".contains("Prog")</code>	true
<code>indexOf(str)</code>	Vị trí xuất hiện đầu tiên	<code>"Java".indexOf('v')</code>	2
<code>replace(old, new)</code>	Thay thế mọi ký tự khớp	<code>"Java".replace('a', 'o')</code>	Jovo
<code>toUpperCase()</code>	Chuyển thành chữ in hoa	<code>"Java".toUpperCase()</code>	JAVA
<code>trim()</code>	Xóa khoảng trắng ở hai đầu	<code>" Java ".trim()</code>	"Java"

💡 Góc phỏng vấn

- String là bất biến và được lưu trong String Pool.
- Dùng StringBuilder để có hiệu năng tốt hơn.
- Dùng StringBuffer khi cần an toàn đa luồng.

★ Thực hành tốt

- Ưu tiên StringBuilder thay vì String khi nối chuỗi trong vòng lặp.
- Tránh tạo ra các đối tượng String không cần thiết.
- Đặt tên biến có ý nghĩa để dễ đọc.

Lớp & Đối tượng trong Java

9

Java là ngôn ngữ lập trình hướng đối tượng.
Mọi thứ trong Java đều là **Đối tượng**.

① Lớp (Class)

- Một **Class** là bản thiết kế (blueprint) hay khuôn mẫu.
- Nó định nghĩa thuộc tính (fields) và hành vi (methods) của đối tượng.
- Bản thân class không chiếm bộ nhớ.

Ví dụ: Định nghĩa một class

```
class Student {  
    String name;           // thuộc tính  
    int age;               // thuộc tính  
  
    void display() {       // phương thức  
        System.out.println(name + " - " + age);  
    }  
}
```

② Đối tượng (Object)

- Một **Object** là một thực thể trong thế giới thật.
- Nó là một thể hiện (instance) của một Class.
- Đối tượng chiếm bộ nhớ trên vùng **Heap**.

Ví dụ: Các đối tượng của class Student

s1 (Object)

name → "An"
age → 20

s2 (Object)

name → "Bình"
age → 22

Cả hai đối tượng đều là thể hiện của class **Student**

③ Thuộc tính (Fields)

- Fields biểu diễn trạng thái hay dữ liệu của đối tượng.
- Mỗi đối tượng có bản sao thuộc tính của riêng nó.

④ Phương thức (Methods)

- Methods biểu diễn hành vi của đối tượng.
- Methods thao tác trên fields và thực hiện hành động.
- Methods được định nghĩa bên trong class.

⑤ Tạo đối tượng

- Đối tượng được tạo bằng từ khóa **new**.

Cú pháp:

```
ClassName objectName = new ClassName();
```

Ví dụ:

```
Student s1 = new Student(); // s1 là một đối tượng  
Student s2 = new Student(); // s2 là đối tượng khác
```

⑥ Truy cập thành phần

- Dùng toán tử chấm (.) để truy cập fields và methods của đối tượng.

Ví dụ:

```
Student s1 = new Student();  
s1.name = "An";           // truy cập field  
s1.age = 20;               // truy cập field  
s1.display();              // gọi phương thức
```

⑦ Class, Đối tượng & Bộ nhớ

Class (Bản thiết kế)

Student

- name : String
- age : int
+ display() : void

new

→

Đối tượng (Instances)

s1

name → "An"
age → 20

s2

name → "Bình"
age → 22

→

Lưu trên vùng Heap

Heap
Memory



Góc phỏng vấn

- Class là khuôn mẫu, Object là thể hiện.
- Từ một class có thể tạo ra nhiều đối tượng.
- Bộ nhớ cho đối tượng được cấp phát trên Heap.



Thực hành tốt

- Thiết kế class bám theo thực thể ngoài đời thật.
- Đề fields là private và cung cấp method public (đóng gói).
- Đặt tên có ý nghĩa cho class, field và method.

1 Constructor

- **Constructor** là một phương thức đặc biệt dùng để khởi tạo đối tượng.
- Nó có tên trùng với tên class.
- Nó không có kiểu trả về (kể cả void).
- Nó được gọi tự động khi đối tượng được tạo.

Cú pháp:

```
class ClassName {  
    ClassName() {  
        // code khởi tạo  
    }  
}
```

Ghi chú

Constructor được dùng để thiết lập giá trị ban đầu cho các thuộc tính của đối tượng.

2 Constructor mặc định

- Là constructor không có tham số.
- Nếu ta không viết constructor nào, Java tự cung cấp một constructor mặc định.
- Nó khởi tạo đối tượng với các giá trị mặc định.

Ví dụ:

```
class Student {  
    String name;  
    int age;  
  
    // Constructor mặc định (do Java cung cấp)  
    Student() { }  
}
```

3 Constructor có tham số

- Là constructor nhận tham số đầu vào.
- Dùng để khởi tạo đối tượng với giá trị cụ thể.

Ví dụ:

```
class Student {  
    String name;  
    int age;  
  
    Student(String name, int age) {  
        this.name = name;  
        this.age = age;  
    }  
}
```

4 Từ khóa: this

- **this** là biến tham chiếu trỏ tới đối tượng hiện tại.
- Dùng để phân biệt biến thể hiện với biến cục bộ.

Ví dụ:

```
class Student {  
    String name;  
    int age;  
    Student(String name, int age) {  
        this.name = name; // trỏ tới biến thể hiện  
        this.age = age;   // trỏ tới biến thể hiện  
    }  
}
```

5 Từ khóa: static

- Thành phần **static** thuộc về Class, không thuộc đối tượng nào.
- Có thể truy cập thông qua tên class.
- Được chia sẻ chung cho mọi đối tượng.

Ví dụ:

```
class Student {  
    static int count = 0; // biến của lớp  
    String name;  
    Student(String name) {  
        this.name = name;  
        count++;          // chung cho mọi đối tượng  
    }  
}
```

6 Từ khóa: final

- Biến **final**: giá trị không thể thay đổi.
- Phương thức **final**: không thể ghi đè (override).
- Class **final**: không thể kế thừa.

Ví dụ:

```
class Student {  
    final String COLLEGE = "ABC Institute"; // hằng số  
  
    final void show() {  
        System.out.println("Không thể ghi đè");  
    }  
}
```

Ghi nhớ

- Tên constructor phải trùng tên class.
- Constructor không có kiểu trả về.
- Nếu không khai báo, JVM tạo constructor mặc định.

★ So sánh nhanh

Từ khóa	Công dụng	Thuộc về	Thay đổi được?
this	Trỏ tới đối tượng hiện tại	Đối tượng	Có
static	Thành phần cấp độ lớp	Class	Chỉ một lần
final	Hằng số / chặn thay đổi & kế thừa	Tùy (biến / method / class)	Không

★ Thực hành tốt

- Dùng **this** để tránh nhập nhằng giữa các biến.
- Dùng **static** cho dữ liệu dùng chung và phương thức tiện ích.
- Dùng **final** để code an toàn và bất biến hơn.

① Đóng gói (Encapsulation)

- Đóng gói là việc gom dữ liệu (fields) và các phương thức xử lý dữ liệu đó vào chung một đơn vị (class).
- Nó hạn chế truy cập trực tiếp vào một số thành phần của đối tượng — đây là cách bảo vệ dữ liệu.

★ Lợi ích

- Che giấu dữ liệu
- Tăng tính bảo mật
- Kiểm soát dữ liệu tốt hơn
- Linh hoạt hơn khi thay đổi

② Getter & Setter

- Dùng để đọc và cập nhật các field **private**.
- Getter trả về giá trị.
- Setter kiểm tra rồi gán giá trị.

Ví dụ:

```
class Employee {  
    private String name;  
  
    public String getName() {  
        return name;  
    }  
  
    public void setName(String name) {  
        this.name = name;  
    }  
}
```

③ Kế thừa (Inheritance)

- Kế thừa là cơ chế cho phép một class nhận lại (kế thừa) thuộc tính và hành vi của class khác.
- Giúp tái sử dụng code và tạo quan hệ IS-A.

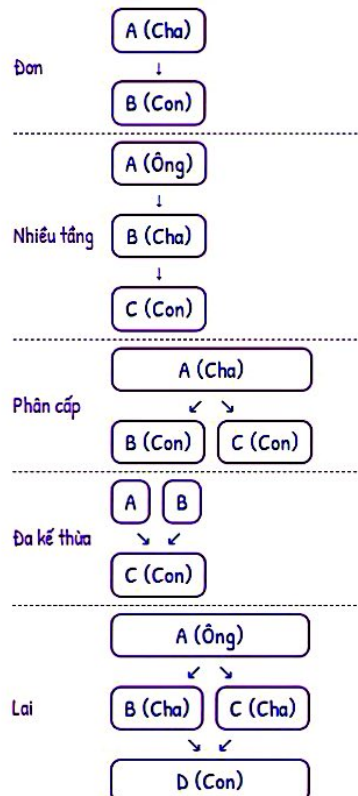
💡 Quan hệ IS-A

Nếu class B kế thừa class A thì B IS-A A.

Ví dụ: Dog IS-A Animal

④ Các loại kế thừa

- Đơn (Single) – một class con kế thừa từ một class cha.
- Nhiều tầng (Multilevel) – con kế thừa cha, rồi lại có class khác kế thừa con đó.
- Phân cấp (Hierarchical) – nhiều class con cùng kế thừa một class cha.
- Đa kế thừa (Multiple) – một class con kế thừa nhiều class cha. (Java không hỗ trợ với class, chỉ hỗ trợ qua Interface)
- Lai (Hybrid) – kết hợp từ hai loại trở lên.



⑤ Từ khóa super

- **super** là biến tham chiếu trỏ tới đối tượng của class cha trực tiếp.
- Dùng để truy cập thành phần của class cha (fields, methods, constructors).

Ví dụ:

```
class Animal {  
    String type = "Animal";  
    Animal() { System.out.println("Animal  
Constructor"); }  
}  
class Dog extends Animal {  
    String type = "Dog";  
    Dog() {  
        super(); // gọi constructor cha  
        System.out.println("Dog Constructor");  
    }  
    void show() {  
        System.out.println(super.type); // field của  
cha  
    }  
}
```

super có thể dùng để:

- ✓ Truy cập field của class cha: `super.fieldName`
- ✓ Gọi method của class cha: `super.methodName()`
- ✓ Gọi constructor của class cha: `super();`

💡 Góc phỏng vấn

- ✓ Đóng gói bảo vệ dữ liệu bằng private + getter/setter.
- ✓ Kế thừa mô hình hóa phân cấp thực tế qua quan hệ IS-A.
- ✓ Ưu tiên composition thay vì kế thừa quá sâu.

★ Thực hành tốt

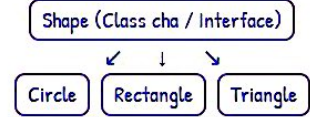
- ✓ Khai báo field là private, cung cấp method truy cập public.
- ✓ Đặt tên có ý nghĩa cho class và method.
- ✓ Dùng `super()` trong constructor con để khởi tạo phần của cha.
- ✓ Ưu tiên Interface thay vì đa kế thừa bằng class.

① Đa hình (Polymorphism)

- Polymorphism nghĩa là "nhiều hình thái".
- Cho phép đối tượng của các class khác nhau được xử lý như đối tượng của một class cha chung.
- Đạt được nhờ **Method Overloading** (lúc biên dịch) và **Method Overriding** (lúc chạy).

💡 Ý tưởng cốt lõi

Cùng một tên phương thức, hành vi khác nhau.
Một interface, nhiều cách cài đặt.



Các class khác nhau, cùng hành vi (draw())

② Đa hình lúc biên dịch

- Đạt được nhờ **Method Overloading**.
- Cùng tên phương thức, khác tham số.
- Được trình biên dịch xác định ngay lúc biên dịch.

Ví dụ:

```
class Calculator {
    int add(int a, int b) { return a + b; }
    int add(int a, int b, int c) { return a + b + c; }
    double add(double a, double b) { return a + b; }
}
```

③ Đa hình lúc chạy

- Đạt được nhờ **Method Overriding**.
- Cùng tên và cùng chữ ký ở class cha và class con.
- Được **JVM** quyết định lúc chạy (dynamic dispatch).

Ví dụ:

```
class Animal {
    void sound() { System.out.println("Tiếng động vật"); }
}
class Dog extends Animal {
    @Override
    void sound() { System.out.println("Gâu gâu"); }
}
Animal a = new Dog(); // tham chiếu kiểu cha
a.sound();             // Gâu gâu (đa hình lúc chạy)
```

④ Lớp trừu tượng (Abstract Class)

- Có thể chứa cả phương thức trừu tượng lẫn phương thức cụ thể.
- Không thể tạo đối tượng trực tiếp.
- Dùng để cung cấp phần nền chung và cài đặt một phần.

Ví dụ:

```
abstract class Shape {
    abstract double area(); // phương thức trừu tượng
    void display() {        // phương thức cụ thể
        System.out.println("Đây là một hình");
    }
}
```

⑤ Interface

- Chỉ chứa phương thức trừu tượng (trước Java 8).
- Từ Java 8, có thể có phương thức default và static.
- Trừu tượng hoàn toàn = đa kế thừa về kiểu.

Ví dụ:

```
interface Drawable {
    void draw(); // phương thức trừu tượng
    default void info() { // default (Java 8+)
        System.out.println("Drawable interface");
    }
}
```

⑥ Ghi đè phương thức (Overriding)

- Class con cung cấp cách cài đặt riêng cho phương thức của class cha.
- Phải cùng tên, cùng tham số và cùng kiểu trả về.
- Mức truy cập không được hẹp hơn class cha.

```
class Parent {
    void show() { System.out.println("Parent"); }
}
class Child extends Parent {
    @Override
    void show() { System.out.println("Child"); }
}
```

★ So sánh nhanh

Tiêu chí	Đa hình lúc biên dịch	Đa hình lúc chạy
Tên gọi khác	Static Polymorphism	Dynamic Polymorphism
Đạt được nhờ	Method Overloading	Method Overriding
Thời điểm liên kết	Lúc biên dịch	Lúc chạy
Chữ ký phương thức	Khác nhau	Giống nhau
Cần kế thừa?	Không bắt buộc	Bắt buộc
Ví dụ	add(int, int)	sound() ở cha & con

💡 Góc phỏng vấn

- Overloading là chuyện của tham số.
- Overriding là chuyện của kế thừa.
- Ưu tiên interface khi định nghĩa "hợp đồng".

★ Thực hành tốt

- Dùng trừu tượng để giấu cài đặt, chỉ lộ hành vi.
- Lập trình hướng interface, không hướng cài đặt cụ thể.
- Luôn dùng annotation **@Override** khi ghi đè.

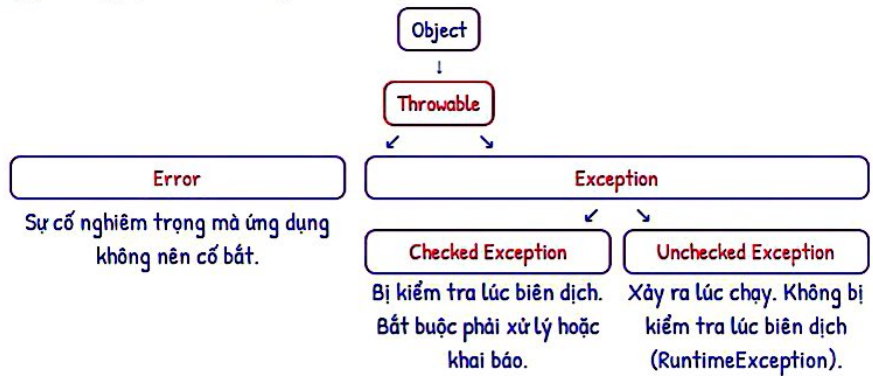
① Ngoại lệ là gì?

Một **Exception** là sự kiện không mong muốn xảy ra và làm gián đoạn luồng chạy bình thường của chương trình.

★ Lợi ích

- Tăng độ tin cậy của code
- Tránh chương trình dừng đột ngột
- Tách riêng phần xử lý lỗi
- Hỗ trợ debug tốt hơn

② Cây phân cấp Exception



③ try - catch - finally

- **try**: chứa đoạn code có rủi ro.
- **catch**: xử lý ngoại lệ.
- **finally**: luôn được chạy (tùy chọn).

```
try {
    int result = 10/0; // code rủi ro
} catch (ArithmeticException e) {
    System.out.println("Không thể chia cho 0");
} finally {
    System.out.println("Khối finally luôn chạy");
}
```

④ Từ khóa throw

- Dùng để chủ động ném ra một ngoại lệ.
- Có thể ném bất kỳ đối tượng nào là lớp con của **Throwable**.

Vi dụ:

```
public void validate(int age) {
    if (age < 18) {
        throw new
        IllegalArgumentException
        ("Tuổi phải từ 18 trở lên");
    }
}
```

⑤ Từ khóa throws

- Dùng trong chữ ký phương thức để khai báo ngoại lệ.
- Chuyển trách nhiệm xử lý cho nơi gọi.

Vi dụ:

```
public void readFile() throws
IOException {
    FileReader fr = new FileReader
    ("file.txt");
}
```

⑥ Ngoại lệ tự định nghĩa

- Ta có thể tự tạo ngoại lệ riêng bằng cách kế thừa class **Exception**.

Vi dụ:

```
class InvalidAgeException extends Exception {
    public InvalidAgeException(String msg) {
        super(msg);
    }
}
// Cách dùng
if (age < 0) {
    throw new InvalidAgeException("Tuổi không thể âm");
}
```

⑦ Các ngoại lệ thường gặp

Exception	Mô tả	Loại
ArithmeticException	Chia cho 0, phép toán không hợp lệ	Unchecked
NullPointerException	Gọi method trên đối tượng null	Unchecked
ArrayIndexOutOfBoundsException	Chỉ số mảng không hợp lệ	Unchecked
NumberFormatException	Chuyển String sang số không hợp lệ	Unchecked
IOException	Lỗi liên quan tới nhập / xuất	Checked
ClassNotFoundException	Không tìm thấy class lúc chạy	Checked

💡 Góc phỏng vấn

- Luôn ưu tiên bắt ngoại lệ cụ thể thay vì Exception chung chung.
- Dùng finally để giải phóng tài nguyên (DB, file, stream).

★ Thực hành tốt

- Không bao giờ bỏ qua ngoại lệ.
- Ghi log thông báo lỗi có ý nghĩa.
- Không bắt Exception nếu không thực sự cần.
- Dùng ngoại lệ tự định nghĩa cho quy tắc nghiệp vụ.

Package & Access Modifier trong Java 14

① Package

- Một **Package** là không gian tên (namespace) giúp tổ chức các class và interface liên quan.
- Giúp tránh trùng tên và dễ bảo trì code hơn.
- Package có sẵn: `java.lang`, `java.util`, `java.io`, v.v.

★ Lợi ích

- ✓ Tránh trùng tên class
- ✓ Tổ chức code tốt hơn
- ✓ Dễ bảo trì
- ✓ Kiểm soát quyền truy cập

② Tạo & dùng Package

Cấu trúc thư mục

```
com
├── qa
│   └── insights
│       ├── Student.java
│       └── MainApp.java
```

Khai báo package (trong Student.java)

```
package com.qa.insights;
```

Import một class

```
import com.qa.insights.Student;
```

③ Câu lệnh import

- Dùng để sử dụng class từ package khác.

Các kiểu import:

- Import một class cụ thể

```
import java.util.ArrayList;
```

- Import theo nhóm (on-demand)

```
import java.util.*;
```

- Static import (Java 5+)

```
import static java.lang.Math.*;
```

💡 Ghi chú

Các class thuộc package `java.lang` được import tự động.

④ Access Modifier (phạm vi truy cập)

Modifier	Trong cùng class	Trong cùng package	Class con (package khác)	Ngoài package	Phù hợp nhất cho
<code>public</code>	Có	Có	Có	Có	Truy cập tối đa
<code>private</code>	Có	Không	Không	Không	Che giấu dữ liệu
<code>protected</code>	Có	Có	Có	Không	Kế thừa
<code>default</code> (không ghi gì)	Có	Có	Không	Không	Truy cập cấp package

⑤ Ví dụ

```
// Public - truy cập được ở mọi nơi
public class PublicClass {}

// Private - chỉ trong class
class PrivateClass {
    private int secret = 10;
}
```

```
// Protected - trong package
// và ở các class con
class Parent {
    protected void show() {}
}

// Default - chỉ trong package
class DefaultClass {}
```

⑥ Truy cập trong các tình huống

Tình huống	public	protected	default	private
Cùng class	✓	✓	✓	✓
Cùng package	✓	✓	✓	✗
Class con (package khác)	✓	✓	✗	✗
Package khác (không kế thừa)	✓	✗	✗	✗

💡 Góc phỏng vấn

- Luôn ưu tiên nguyên tắc "quyền tối thiểu".
- Dùng `private` cho field và cung cấp getter/setter.
- Nên cẩn trọng khi dùng quyền truy cập default.

★ Thực hành tốt

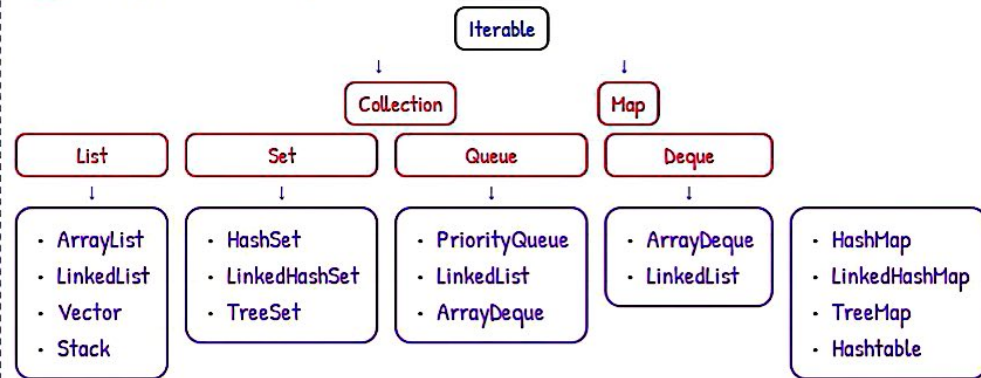
- Gom các class liên quan vào package có ý nghĩa.
- Đặt tên package rõ ràng (vd: `com.company.module`).
- Tránh import kiểu wildcard (*), hãy import cụ thể.

① Collection Framework là gì?

Collection Framework trong Java cung cấp một bộ các class và interface dựng sẵn để lưu trữ, thao tác và truy xuất các nhóm đối tượng một cách hiệu quả.

Nó nằm trong package `java.util`.

② Cây phân cấp Collection



③ List vs Set vs Queue vs Map

Tiêu chí	List	Set	Queue	Map
Định nghĩa	Tập hợp có thứ tự các phần tử.	Tập hợp các phần tử duy nhất.	Xử lý phần tử theo thứ tự FIFO.	Tập hợp các cặp key - value.
Trùng lặp	Cho phép	Không cho phép	Cho phép	Key: Không, Value: Có
Thứ tự	Giữ đúng thứ tự thêm vào.	HashSet - không thứ tự LinkedHashSet - thứ tự thêm vào TreeSet - đã sắp xếp	FIFO (vào trước ra trước)	Không thứ tự (HashMap) Thứ tự thêm vào (LinkedHashMap) Đã sắp xếp (TreeMap)
Cách truy cập	Theo chỉ số (get/set)	Không truy cập theo chỉ số	Theo đầu/cuối (poll/peek)	Theo key (get/put)
Class tiêu biểu	ArrayList, LinkedList, Vector, Stack	HashSet, LinkedHashSet, TreeSet	PriorityQueue, ArrayDeque, LinkedList	HashMap, LinkedHashMap, TreeMap, Hashtable

④ Khi nào dùng loại nào?

List

- Khi thứ tự quan trọng.
- Khi cho phép phần tử trùng.
- Khi cần truy cập theo chỉ số.

Ví dụ:

Danh sách sinh viên, tên, sản phẩm.

Set

- Khi cần tính duy nhất.
- Khi không cần truy cập theo chỉ số.

Ví dụ:

Lưu ID duy nhất, email, URL đã truy cập.

Queue

- Khi phần tử cần xử lý theo thứ tự FIFO.
- Dùng trong lập lịch, bộ đệm, hàng đợi công việc.

Ví dụ:

Hàng đợi máy in, hàng đợi tác vụ.

Map

- Khi cần lưu dữ liệu theo cặp key - value.
- Khi cần tra cứu nhanh theo key.

Ví dụ:

userId → user, countryCode → country.

⑤ Sơ đồ tổng quan



★ Ghi nhớ

- ✓ Collection → các phần tử đơn lẻ.
- ✓ Map → các cặp key - value.
- ✓ Chọn đúng collection để code chạy nhanh và sạch hơn.

💡 Góc phỏng vấn

Hãy nắm rõ sự khác nhau, tình huống sử dụng và độ phức tạp thời gian của từng class collection.

📝 Ghi chú

Mọi class collection đều làm việc với đối tượng (kiểu tham chiếu), không dùng trực tiếp kiểu nguyên thủy.

Interface **Map** trong **java.util** lưu dữ liệu dưới dạng cặp **key - value**.
Mỗi key phải là duy nhất và ánh xạ tới đúng một value.

① Các cài đặt của Map

HashMap

- Cài đặt Map phổ biến nhất.
- Không có thứ tự, cho phép một key null và nhiều value null.
- Không đồng bộ.

LinkedHashMap

- Giữ thứ tự thêm vào nhờ danh sách liên kết đôi.
- Cho phép key null và value null.
- Chậm hơn HashMap một chút.

TreeMap

- Lưu key theo thứ tự đã sắp xếp (thứ tự tự nhiên).
- Không cho phép key null.
- Chậm hơn HashMap.

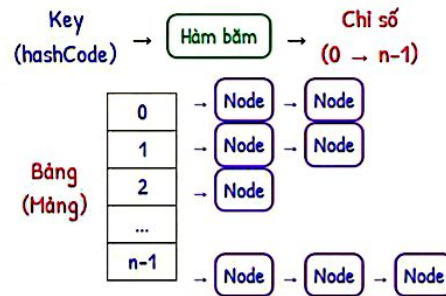
Hashtable

- Class cũ (từ Java 1.0).
- Đồng bộ và an toàn đa luồng.
- Không cho phép key null lẫn value null.

② HashMap hoạt động bên trong

- HashMap dùng bảng băm (mảng các bucket).
- hashCode() của key được dùng để tìm chỉ số.
- Va chạm được xử lý bằng Linked List (hoặc cây Tree từ Java 8+).

Cấu trúc bên trong của HashMap



③ Bảng so sánh

Tiêu chí	HashMap	LinkedHashMap	TreeMap	Hashtable
Thứ tự	Không có thứ tự	Thứ tự thêm vào	Sắp xếp theo key (tự nhiên / Comparator)	Không có thứ tự
Key null	Cho phép (1 key null)	Cho phép (1 key null)	Không cho phép	Không cho phép
Value null	Cho phép	Cho phép	Cho phép	Không cho phép
An toàn đa luồng	Không	Không	Không	Có (đồng bộ)
Hiệu năng	Nhanh nhất	Chậm hơn HashMap chút ít	Chậm (thao tác cây $O(\log n)$)	Chậm (do đồng bộ)
Cấu trúc dữ liệu bên dưới	Hash Table + Linked List / Tree	Hash Table + Doubly Linked List	Cây đỏ đen (Red-Black Tree)	Hash Table + Linked List
Class	java.util.HashMap	java.util.LinkedHashMap	java.util.TreeMap	java.util.Hashtable



Góc phỏng vấn

- Dùng HashMap khi không quan tâm thứ tự và cần hiệu năng tốt nhất.
- Dùng LinkedHashMap khi cần giữ thứ tự thêm vào.
- Dùng TreeMap khi cần key đã sắp xếp.
- Tránh dùng Hashtable trong ứng dụng hiện đại.



Thực hành tốt

- Ưu tiên HashMap thay vì Hashtable.
- Nếu key cần có thứ tự, dùng TreeMap.
- Nếu cần thứ tự thêm vào, dùng LinkedHashMap.
- Tránh dùng đối tượng có thể thay đổi làm key.



Ví dụ HashMap

```

Map<String, Integer> map = new HashMap<>();
map.put("A", 10);
map.put("B", 20);
map.put(null, 30); // cho phép key null
map.put("A", 40); // value được cập nhật
System.out.println(map); // {A=40, B=20, null=30}
    
```

Phương thức thường dùng

put(key, value)	→ Thêm / Cập nhật
get(key)	→ Lấy value
remove(key)	→ Xóa key
containsKey(key)	→ Kiểm tra key
size()	→ Số phần tử

Java cung cấp các class để làm việc với file và lưu trữ đối tượng một cách lâu dài thông qua **Serialization**.

① Các class xử lý file

File Class

- Đại diện cho đường dẫn file hoặc thư mục.
- Cung cấp các method tạo, xóa, kiểm tra tồn tại, v.v.

```
File f = new  
File("test.txt");  
f.createNewFile();  
f.exists();  
f.delete();
```

FileReader

- Đọc dữ liệu ký tự từ file.
- Dùng cho việc nhập ký tự đơn giản.

```
FileReader fr =  
new FileReader("a.txt");  
int ch;  
while((ch = fr.read()) != -1)  
System.out.print((char)ch);  
fr.close();
```

FileWriter

- Ghi dữ liệu ký tự vào file.
- Tự tạo file nếu file chưa tồn tại.

```
FileWriter fw =  
new FileWriter("a.txt");  
fw.write("Hello Java");  
fw.flush();  
fw.close();
```

BufferedReader

- Đọc văn bản hiệu quả theo từng dòng.
- Dùng bộ đệm nội bộ nên nhanh hơn.

```
BufferedReader br = new  
BufferedReader(new  
FileReader("a.txt"));  
String line;  
while((line = br.readLine())  
!= null)  
System.out.println(line);  
br.close();
```

BufferedWriter

- Ghi văn bản hiệu quả theo từng dòng.
- Dùng bộ đệm nội bộ nên nhanh hơn.

```
BufferedWriter bw = new  
BufferedWriter(new  
FileWriter("a.txt"));  
bw.write("Hello Java");  
bw.newLine();  
bw.close();
```

② Serialization

Serialization là quá trình chuyển trạng thái của đối tượng thành một luồng byte, để có thể lưu vào file hoặc gửi qua mạng.

Các bước:

1. Đối tượng phải implement interface **Serializable**.
2. Dùng **ObjectOutputStream** để ghi đối tượng ra file.

```
class Student implements Serializable {  
String name; int age;  
Student(String n, int a){ name=n; age=a; }  
}  
Student s = new Student("Minh", 25);  
FileOutputStream fos = new  
FileOutputStream("student.ser");  
ObjectOutputStream oos = new  
ObjectOutputStream(fos);  
oos.writeObject(s); // serialization  
oos.close(); fos.close();
```

Đối tượng

Student
(name, age)



Serialization



File
(student.ser)

③ Deserialization

Deserialization là quá trình ngược lại của Serialization. Nó chuyển luồng byte trở lại thành đối tượng.

Các bước:

1. Dùng **ObjectInputStream** để đọc đối tượng từ file.
2. Ép kiểu đối tượng về đúng class ban đầu.

```
FileInputStream fis = new FileInputStream("student.ser");  
ObjectInputStream ois = new ObjectInputStream(fis);  
Student s2 = (Student) ois.readObject(); // deserialization  
System.out.println(s2.name + " - " + s2.age);  
ois.close(); fis.close();
```

File
(student.ser)

Deserialization



Student
(name, age)

④ Điểm chính

- Class **File** dùng cho thao tác file & thư mục.
- Dùng class có bộ đệm (**Buffered**) để hiệu năng tốt hơn.
- **Serialization** giúp lưu lại trạng thái đối tượng.
- **Deserialization** tái tạo đối tượng từ trạng thái đã lưu.
- Field **transient** và **static** không được **serialize**.

Góc phỏng vấn

- Khác nhau giữa **File** và **Path**?
- Khi nào dùng **BufferedReader** thay vì **FileReader**?
- Vì sao cần **serialization**?
- Điều gì xảy ra nếu **serialVersionUID** thay đổi?
- Khác nhau giữa **Serializable** và **Externalizable**?

✓ Thực hành tốt

- Luôn đóng stream trong khối **finally** hoặc dùng **try-with-resources**.
- Không **serialize** khi không thực sự cần.
- Dùng stream có bộ đệm cho file lớn.
- Kiểm tra đường dẫn file và xử lý ngoại lệ cẩn thận.

★ Ghi nhớ

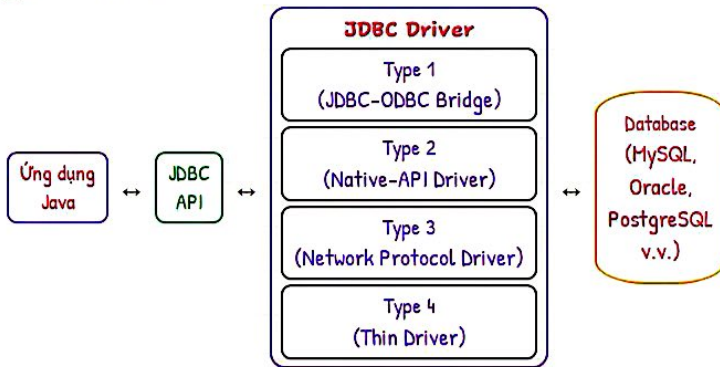
Serialization = Đối tượng → Luồng byte (Lưu)
Deserialization = Luồng byte → Đối tượng (Nạp)

⚠ Các ngoại lệ thường gặp

- **FileNotFoundException** - không tìm thấy file.
- **IOException** - lỗi nhập/xuất chung.
- **ClassNotFoundException** - không tìm thấy class khi **deserialize**.

JDBC (Java Database Connectivity) là API cho phép ứng dụng Java kết nối và tương tác với cơ sở dữ liệu.

① Kiến trúc JDBC



② DriverManager

- Class DriverManager quản lý danh sách các driver cơ sở dữ liệu.
- Được dùng để thiết lập kết nối.
- Nó nạp driver phù hợp và tạo ra kết nối.

```
Connection conn = DriverManager.getConnection(
    "jdbc:mysql://localhost:3306/qa_db",
    "root", "password");
```

③ Connection

- Interface Connection đại diện cho một phiên làm việc với cơ sở dữ liệu.
- Được tạo ra thông qua DriverManager.
- Dùng để tạo Statement, PreparedStatement hoặc CallableStatement.

```
Connection conn = DriverManager.getConnection(
    "jdbc:mysql://localhost:3306/qa_db", "root",
    "password");
// Đừng quên đóng kết nối
```

④ Statement

- Statement dùng để thực thi các câu lệnh SQL đơn giản.
- Không an toàn trước SQL Injection.
- Phù hợp với các câu SQL tĩnh.

```
Statement stmt = conn.createStatement();
String sql = "SELECT * FROM users";
ResultSet rs = stmt.executeQuery(sql);
```

⑤ PreparedStatement

- Câu lệnh SQL đã được biên dịch trước.
- Truy vấn có tham số.
- Chống SQL Injection và cải thiện hiệu năng.

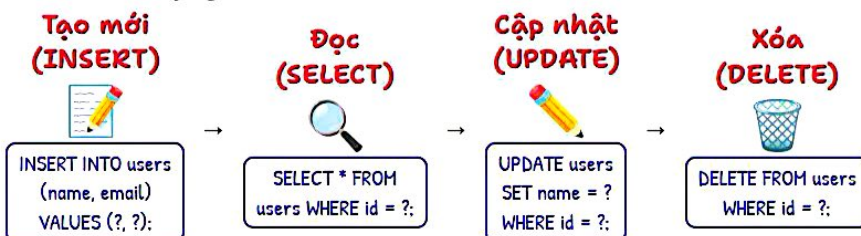
```
PreparedStatement ps = conn.prepareStatement(
    "SELECT * FROM users WHERE id = ?");
ps.setInt(1, 101);
ResultSet rs = ps.executeQuery();
```

⑥ ResultSet

- ResultSet đại diện cho kết quả trả về của một truy vấn.
- Cung cấp con trỏ để duyệt qua từng dòng.

```
while (rs.next()) {
    int id = rs.getInt("id");
    String name = rs.getString("name");
    System.out.println(id + " - " + name);
}
```

⑦ Sơ đồ quy trình CRUD



💡 Điểm chính

- Nạp driver (tự động từ JDBC 4+).
- Lấy một Connection.
- Tạo Statement / PreparedStatement.
- Thực thi câu truy vấn SQL.
- Xử lý ResultSet.
- Đóng tài nguyên.

🔗 Góc phỏng vấn

- JDBC là gì? → API kết nối Java với DB.
- Khác nhau giữa Statement & PreparedStatement?
- Vì sao phải đóng Connection, Statement, ResultSet?

✅ Thực hành tốt

- Dùng PreparedStatement để an toàn & hiệu năng tốt.
- Luôn đóng tài nguyên (try-with-resources).
- Xử lý ngoại lệ một cách rõ ràng, có ý nghĩa.

⚠️ Lỗi thường gặp

- Quên đóng tài nguyên.
- Dùng Statement với dữ liệu người dùng nhập (SQL Injection).
- Không xử lý SQLException.